Paola Cherneva
h12319305

PROJECT REPORT

1. MODELS

BASELINE MODEL

The model used for the baseline was Meta's Llama-3.1-8B-Instruct loaded in 4-bit quantization through BitsAndBytes. To actually access the model, you first need to get a license for it through HuggingFace, and then use your HuggingFace token, which, in my case, was stored as a secret in Google Colab to avoid having to enter it manually each session.

The model was given a German system prompt instructing it to answer Austrian tax questions concisely and factually, without fabricating legal sources. Instruction-tuned models like Llama expect a “chat format” - the system message acts as the prompt, and the user message contains the actual question to answer.

Inference used greedy decoding with max_new_tokens=150. Greedy made sense here because, for factual legal questions, you want the most probable answer - sampling could easily lead to hallucinated paragraph numbers or made-up provisions. The 150 token limit was enough for full answers without making inference too slow across all 644 questions.

This model was already pre-trained by Meta on a huge multilingual corpus, so no weights were modified - this was just to test and see the model's existing knowledge.

The dataset files were stored on Google Drive and loaded at the start of each notebook, because Colab deletes everything when a session ends, so using Google Drive helped to keep things persistent. Baseline results, as all other results, were saved back to Google Drive, so they can be used in the subsequent notebooks for comparison.

FINE-TUNED MODEL

The initial idea was to fine-tune the same 8B model used for the baseline, but that just didn't work. Even with 4-bit quantization and LoRA, the T4 ran out of memory. After several failed attempts, the only realistic option was to drop down to Llama-3.2-1B-Instruct (4-bit, Unsloth). Unsloth is a library that optimizes LLM fine-tuning to use less memory and run faster - it's what made it possible to fine-tune even the 1B model on a T4 without running out of VRAM. That's obviously a much weaker model, but it was the only thing that was possible. Even then, the full training run took around 4 hours on Colab.

LoRA adapters were applied with rank 16 and alpha 16 - the rank controls how large those matrices are (higher = more capacity to learn, but more memory), and alpha controls how strongly the updates are applied to the original weights. With alpha and rank both at 16, the scaling factor is 1.0, so the LoRA updates apply at full strength. All major layers were targeted (attention and MLP projections) since the model needs to shift both how it processes questions and what factual knowledge it draws on. Dropout was set to 0.05 to reduce overfitting given the small training set.

Building the fine-tuning dataset was one of the hardest parts of this whole project. The source materials came mainly from KStG §§ 7, 8, 10, EStG §§ 22, 23, 25, UStG §§ 1, 6, 11, 12, BAO §§ 119, 131, 132, 138, 143.

Paola Cherneva
h12319305

The problem is that for fine-tuning, the quality of the data really matters. So at the end, 60 question-answer pairs was what I could realistically create with reasonable confidence in the quality. Hundreds of expert-verified examples would have been much better, but creating those manually within the scope of this assignment simply wasn't doable.

The 60 examples were formatted in the Llama 3 chat template and trained with SFTTrainer from TRL: 1 epoch (one pass through the data), batch size 2 (two examples processed at a time), gradient accumulation 4 (gradients are summed over 4 batches before updating weights, giving an effective batch size of 8), learning rate 2e-4 (how aggressively weights get updated each step), AdamW 8-bit, fp16, 5 warmup steps, packing enabled. Max sequence length was 512 tokens - packing means short examples get concatenated into one 512-token sequence, so you don't waste tokens on padding. The trained LoRA weights were saved to Google Drive afterwards. During inference, partial results were saved every 50 questions just in case Colab crashes.

Generation at inference time used max_new_tokens=80 with greedy decoding, which in the end turned out to be too low.

RAG MODEL

The RAG pipeline used the same Llama-3.2-1B-Instruct (4-bit, Unsloth). As a knowledge base, I used 22 PDF documents containing Austrian tax legislation - the same sources we had to use for creating the initial dataset. These were downloaded from the RIS website and JUSLINE, then stored in a folder on Google Drive.

Text extraction was done with pypdf. The text was then chunked into 300-word pieces with 50-word overlap. 300 words was somewhere in the middle - smaller chunks lose paragraph-level context, bigger ones get too broad for precise retrieval. The overlap makes sure that if something important falls right at a chunk boundary, it still shows up in at least one chunk. This produced 476 chunks total.

For embedding and retrieval, paraphrase-multilingual-MiniLM-L12-v2 from sentence-transformers was used, because it supports German very well. All 476 chunks were embedded and indexed with FAISS, which finds the most similar chunks to a given question based on cosine similarity. At query time, each question was embedded with the same model, and the top 3 closest chunks were retrieved. These chunks were then placed into the prompt alongside a German system message and the question. Generation used max_new_tokens=120, greedy decoding, batch size 4, and a max_seq_length of 1024 tokens.

3. EVALUATION

All three models were evaluated against the “correct” answers from our initial dataset using two different metrics: **ROUGE-L and BERTScore (F1).**

Main Results Table

Model	ROUGE-L	BERTScore (F1)
Baseline (Llama-3.1-8B)	0.1434	0.8258

Paola Cherneva
h12319305

Fine-Tuned (Llama-3.2-1B + LoRA)	0.1451	0.8268
RAG (Llama-3.2-1B)	0.0549	0.7848

The baseline and fine-tuned models performed almost identically, and the RAG clearly performed way worse.

ROUGE-L measures exact word overlap**.** A score of 0.14 (Baseline and Fine-tuning) means only about 14% of the words in the model's answers match the reference answers, which seems reasonable for such a task, because the model can give a correct answer using completely different words than the reference.

The scored 0.05 for the RAG also makes sense, because the model now includes extra information pulled from the retrieved documents in its answers, which lowers the chances of word overlap even more.

BERTScore at ~0.83 confirms that the top two models' answers are semantically close to the references. The RAG model again scored lower here, which tells us that there is something wrong with the pipeline, since that should not be the case.

We can see that BERTScore is the better metric to use for this task. Had I only used ROUGE I would've seriously underestimated the models' performance. The scores show that the models are producing semantically correct answers, which are just phrased differently from the references.

4. ERROR ANALYSIS

1. RAG MODEL

The RAG model produced the worst results, and if we look at the output, we can clearly see that many answers are just fragments, rather than full sentences.

The problem is that the full RAG prompt (system message + 3 chunks + question) was probably using more tokens than the max_seq_length, which was set to 1024. Everything past that got cut off, and since the question sits at the end of the prompt, it was probably the first thing to go. The model never even saw what it was being asked - it just got a bunch of legal text ending mid-sentence and continued from there.

Setting the sequence length to 4096 would have fixed this entirely. The 1024 limit was meant to save memory on the T4, but in practice, it ruined the whole pipeline.

2. FINE-TUNED MODEL

The fine-tuned model performed basically the same as the baseline, even though it's supposed to really outdo it.

The most obvious issue is that 60 examples just aren't enough for such a task.

The 512-token max sequence length again created a truncation problem. Each example packed the system prompt, question, and answer into 512 tokens. The prompt and question already took up a lot of tokens, so longer answers got cut off. The model never saw the full version of those answers, so it learned to produce incomplete responses. A higher sequence length would have helped, but again, Colab's T4 memory

Paola Cherneva
h12319305

constraints were a problem- longer sequences mean more memory per example, and the GPU was already near its limit.

On top of that, 60 examples with 1 epoch and an effective batch size of 8 means roughly 8 gradient updates total. That's not enough to teach the model anything meaningful, but to just slightly mess up what it already knows. More epochs would have been better, but training already took 4 hours and risking a session timeout for maybe better returns didn't seem worth it.

Also, the max_new_tokens=80 limit at inference was too low. Some correct answers need more tokens, so the model couldn't finish longer responses even when it knew the right answer.

The baseline outperforming both more “advanced” models kind of makes sense given the setup. The baseline used an 8B-parameter model while the fine-tuned and RAG approaches used 1B - an 8× difference in capacity. The larger model has substantially more knowledge from pre-training, having seen more German text, more legal text, and more structured reasoning examples during training on Meta's corpus.